1. Structure of a Compiler & Phase Interaction

Definition / Theory

A **compiler** is a system software that translates a high-level programming language into equivalent machine-level code. The compilation process is divided into multiple phases, each responsible for a specific task.

Major Phases of a Compiler

1. **Lexical Analysis (Scanner)**
   - Converts source code into tokens.
   - Removes whitespace and comments.
   - Example: `int a = 10;` → `int`, `a`, `=`, `10`, `;`
2. **Syntax Analysis (Parser)**
   - Checks grammatical correctness using CFG (Context-Free Grammar).
   - Produces a **Parse Tree / Syntax Tree**.
3. **Semantic Analysis**
   - Ensures meaning is correct.
   - Type checking, variable declaration, scope resolution.
4. **Intermediate Code Generation**
   - Produces machine-independent code (e.g., three-address code).
5. **Code Optimization**
   - Improves efficiency (removes redundant code, etc.).
6. **Code Generation**
   - Converts intermediate code into machine code.

Neat Diagram of Compiler Phases

```
Source Code
   |
   v
+-------------------+
| Lexical Analysis  |  → Tokens
+-------------------+
   |
   v
+-------------------+
| Syntax Analysis   |  → Parse Tree
+-------------------+
   |
   v
+-------------------+
| Semantic Analysis |  → Annotated Tree
+-------------------+
   |
   v
+-------------------+
| Intermediate Code |
+-------------------+
   |
   v
+-------------------+
| Optimization      |
+-------------------+
   |
   v
+-------------------+
| Code Generation   |
+-------------------+
   |
   v
Target Code
```

Interaction Between Phases

- **Lexical → Syntax**
  - Lexical analyzer provides tokens to the parser.
  - Example: `id + id`
- **Syntax → Semantic**

    - Parser builds structure → Semantic analyzer checks meaning.
    - Example: Type mismatch detection.
- **Semantic → Symbol Table**
    - Maintains variable info (type, scope, memory location).

Key Points for Exam

- Each phase passes output to the next phase.
- Errors are detected at different stages:
    - Lexical → invalid token
    - Syntax → grammar error
    - Semantic → type mismatch
- Phases may interact via **symbol table** and **error handler**.

✅ 2. Role of Finite Automata in Lexical Analysis + DFA Construction

Definition / Theory

A **Finite Automaton (FA)** is a mathematical model used to recognize patterns.
In lexical analysis, FA is used to identify **tokens from input strings** based on regular expressions.

Role of Finite Automata in Lexical Analysis

- Converts **regular expressions → DFA**
- Efficiently scans input in **linear time**
- Recognizes:
    - Identifiers
    - Keywords
    - Numbers
    - Operators

Steps: RE → DFA Construction

Example Regular Expression:

(a|b)*abb

Step 1: Convert RE → NFA (Thompson Construction)

(Conceptual step – you can mention it briefly in exam)

Step 2: Convert NFA → DFA (Subset Construction)

Step 3: Final DFA

DFA Transition Table

State	a	b	Final State
q0	q1	q0	No
q1	q1	q2	No
q2	q1	q3	No
q3	q1	q0	Yes

DFA Diagram (Neat Representation)

```
(q0) --a--> (q1) --b--> (q2) --b--> [q3]
 ^          |          |
 |          a          a
 |___________|__________|
```

- **q0** → Start state
- **q3** → Final (accepting state)

Working (Token Recognition)

Input: `aabb`

- q0 → a → q1
- q1 → a → q1
- q1 → b → q2
- q2 → b → q3 ✅ (Accepted)

Why DFA is Efficient

- No backtracking required
- Deterministic → single path execution
- Time Complexity: **O(n)**

Key Points for Exam

- FA is the backbone of lexical analyzer
- Regular expressions define tokens
- DFA ensures fast and deterministic scanning
- Used in tools like **Lex/Flex**

1. Bootstrapping in Compiler Design

Definition / Theory

Bootstrapping is the process of developing a compiler using the same programming language that the compiler is intended to compile.

In simple terms, a compiler is **self-hosted** when it is written in its own language.

Concept Explanation

- Initially, a simple compiler is written in another language (like assembly or C).
- This compiler is then used to compile a more advanced version of the compiler written in the target language.
- The process continues until the compiler can compile itself.

Types of Bootstrapping

1. **Cross Bootstrapping**
    - Compiler runs on one machine but generates code for another.
2. **Self Bootstrapping**
    - Compiler compiles its own source code.

Bootstrapping Process (Steps)

1. Write a basic compiler in a low-level language.
2. Develop a new compiler in the target language.
3. Use the existing compiler to compile the new compiler.
4. Gradually improve until full self-compilation is achieved.

Advantages

- Helps in **compiler development and testing**
- Improves **portability**
- Enables **language evolution**

Example

- Early versions of **C compiler** were written in assembly, later rewritten in C.

Key Points for Exam

- Bootstrapping = self-compilation concept
- Important for modern compiler construction
- Reduces dependency on other languages over time

✅ 2. Input Buffering Techniques in Lexical Analysis

Definition / Theory

Input buffering is a technique used in lexical analyzers to read source code efficiently by reducing the number of I/O operations.

Instead of reading one character at a time, data is read in blocks into buffers.

Need for Input Buffering

- Reading character-by-character is slow due to frequent disk access
- Improves performance by minimizing I/O calls
- Supports lookahead for token recognition

Types of Buffering Techniques

---

1. Single Buffer Scheme

Working

- Uses one buffer to store input characters
- When buffer is exhausted, it is refilled

Limitations

- Inefficient due to frequent reloads
- Difficult to handle lookahead

---

2. Double Buffer Scheme (Two-Buffer Technique)

Working

- Uses two buffers of equal size
- When one buffer is processed, the other is filled
- Uses **sentinel characters (EOF)** to mark buffer end

Neat Diagram of Double Buffer

```
+-----------+-----------+
| Buffer 1  | Buffer 2  |
+-----------+-----------+
   ^           ^
   |           |
 lexemeBegin    forward
```

Pointers Used

- **lexemeBegin** → Start of token
- **forward** → Moves ahead to scan characters

Advantages

- Reduces I/O operations
- Supports efficient lookahead
- Avoids boundary checking using sentinels

---

3. Sentinel Technique

- Special character (EOF) placed at end of buffer
- Eliminates need to check buffer limits explicitly

Comparison of Buffering Techniques

Feature	Single Buffer	Double Buffer
I/O Efficiency	Low	High

Lookahead Support	Difficult	Easy
Performance	Slow	Fast
Implementation	Simple	Moderate

Impact on Performance

- **Single Buffer**
  - More I/O calls → slower execution
- **Double Buffer**
  - Parallel processing → faster scanning
  - Reduces overhead significantly

Key Points for Exam

- Input buffering improves lexical analyzer efficiency
- Double buffering is widely used in real compilers
- Sentinel reduces boundary checks
- Important for handling large source programs efficiently

1. Design of Lexical Analyzer Generator (LEX)

Definition / Theory

A **Lexical Analyzer Generator (LEX)** is a tool that automatically generates a lexical analyzer (scanner) from a set of **token definitions specified using regular expressions**.

Example tools: **LEX / FLEX**

Structure of a LEX Program

A LEX program is divided into **three sections**:

```
%{
 // C declarations
%}

%%
 // Token patterns (Regular Expressions)
%%

 // User-defined functions
```

Working of LEX

1. User provides **token patterns (REs)**.
2. LEX converts RE → **NFA** → **DFA**.
3. DFA is converted into **C code**.
4. Generated code scans input and returns tokens.

Example Token Specification

```
%%
[0-9]+      { return NUMBER; }
[a-zA-Z]+   { return ID; }
"+"         { return PLUS; }
%%
```

Translation Process (Pattern → Executable Code)

Step 1: Regular Expression

- Example: `[a-zA-Z]+`

Step 2: RE → NFA

- Using Thompson construction

Step 3: NFA → DFA

- Using subset construction

Step 4: DFA → Transition Table

- States and transitions stored

Step 5: Code Generation

- LEX generates C code with:
    - Transition tables
    - State handling logic
    - Token return statements

Internal Working (Simplified Flow)

Regular Expressions
↓
NFA
↓
DFA
↓
Transition Table
↓
Generated C Code (Scanner)

Advantages of LEX

- Automates lexical analyzer generation
- Efficient (uses DFA)
- Reduces manual coding errors

Key Points for Exam

- LEX uses **regular expressions → DFA → C code**
- Generated scanner works in **linear time**
- Widely used with **YACC (parser generator)**

✅ 2. Eliminating Left Recursion & Left Factoring (LL(1) Grammar)

Definition / Theory

- **Left Recursion**: A grammar is left recursive if
  `A → Aα | β`
- **Left Factoring**: Used to remove ambiguity when productions share a common prefix.

These transformations are required to convert grammar into **LL(1)** form for predictive parsing.

Given Grammar (Example)

E → E + T | T
T → T * F | F
F → (E) | id

🔹 Step 1: Eliminate Left Recursion

For E:

E → E + T | T

Using formula:

A → Aα | β ⇒
A → βA'
A' → αA' | ε

Result:

E → T E'
E' → + T E' | ε

For T:

T → T * F | F

Result:

T → F T'
T' → * F T' | ε

F remains same:

F → (E) | id

🔹 Step 2: Left Factoring

Check if any productions have common prefixes.

In this grammar:

- No common prefixes → No need for left factoring

Final LL(1) Grammar

E → T E'
E' → + T E' | ε

T → F T'
T' → * F T' | ε

F → (E) | id

Intermediate Steps Summary

Step	Transformation
1	Remove left recursion from E
2	Remove left recursion from T
3	Check for left factoring

4 Obtain LL(1) grammar

Why This Grammar is LL(1)

- No left recursion
- No ambiguity
- One lookahead symbol is enough
- Suitable for predictive parsing

Key Points for Exam

- Left recursion must be eliminated for LL(1)
- Left factoring removes common prefixes
- Final grammar should be **deterministic**
- Important for **top-down parsers**

1. Lexical Analysis, Token Recognition & Buffering Techniques

Definition / Theory

Lexical Analysis is the first phase of a compiler that converts a stream of input characters into a sequence of **tokens**.
A **token** is a meaningful sequence of characters such as keywords, identifiers, operators, etc.

Process of Lexical Analysis

1. **Input Reading**
    - Source code is read character by character.
2. **Lexeme Formation**
    - Characters are grouped into meaningful units called **lexemes**.

3. **Token Generation**
    - Each lexeme is mapped to a token.
    - Example: `int x = 10;`

Lexeme	Token
int	KEYWORD
x	ID
=	ASSIGN
10	NUMBER

Token Recognition

- Done using **Regular Expressions** and **Finite Automata (DFA)**.
- Each token pattern is defined as a regular expression.

Example:

- Identifier → `[a-zA-Z][a-zA-Z0-9]*`
- Number → `[0-9]+`

Recognition Process

1. Convert RE → DFA
2. Scan input using DFA
3. When a final state is reached → token is recognized

Buffering Techniques

Need

- Improves efficiency by reducing I/O operations.

---

1. Single Buffer

- One buffer used
- Slow due to frequent reloads

---

2. Double Buffer (Two-Buffer Scheme)

```
+-----------+-----------+
| Buffer 1  | Buffer 2  |
+-----------+-----------+
```

- Two buffers used alternately
- Uses **sentinel (EOF)**

Pointers Used

- **lexemeBegin** → start of lexeme
- **forward** → scans ahead

Advantages of Buffering

- Faster input processing
- Supports lookahead
- Reduces disk access

Key Points for Exam

- Lexical analysis converts input → tokens
- Token recognition uses DFA
- Buffering improves performance
- Double buffering is widely used

✅ 2. FIRST & FOLLOW Sets and Predictive Parsing Table

Definition / Theory

- **FIRST(α)**: Set of terminals that begin strings derived from α
- **FOLLOW(A)**: Set of terminals that can appear immediately after A

Used to construct **LL(1) predictive parsing table**.

Given Grammar (Example)

E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id

🔹 Step 1: Compute FIRST Sets

FIRST(F)

F → (E) | id
FIRST(F) = { '(', id }

FIRST(T)

T → F T'
FIRST(T) = FIRST(F) = { '(', id }

FIRST(E)

E → T E'
FIRST(E) = FIRST(T) = { '(', id }

FIRST(E')

E' → + T E' | ε
FIRST(E') = { '+', ε }

FIRST(T')

T' → * F T' | ε
FIRST(T') = { '*', ε }

🔹 Step 2: Compute FOLLOW Sets

Rules:

- $ in FOLLOW(Start symbol)
- If A → αBβ → FIRST(β) added to FOLLOW(B)
- If β → ε → FOLLOW(A) added

FOLLOW(E)

FOLLOW(E) = { ')', $ }

FOLLOW(E')

FOLLOW(E') = { ')', $ }

FOLLOW(T)

FOLLOW(T) = { '+', ')', $ }

FOLLOW(T')

FOLLOW(T') = { '+', ')', $ }

FOLLOW(F)

FOLLOW(F) = { '*', '+', ')', $ }

🔹 Step 3: Predictive Parsing Table

Non-Terminal	id	+	*	(	)	$
E	T E'			T E'		
E'		+ T E'			ε	ε
T	F T'			F T'		
T'		ε	* F T'		ε	ε
F	id			(E)		

Construction Rules

- If `A → α`, for each `a ∈ FIRST(α)` → put A → α in table[A, a]
- If `ε ∈ FIRST(α)` → use FOLLOW(A)

Key Points for Exam

- FIRST → starting terminals
- FOLLOW → what comes after
- Used for LL(1) parsing table
- No multiple entries → grammar is LL(1)

1. Role of Parser and Context-Free Grammars (CFG) in Syntax Analysis

Definition / Theory

Syntax Analysis (Parsing) is the second phase of a compiler that checks whether the sequence of tokens follows the grammatical rules of the language.

A **Parser** is a component that takes tokens from the lexical analyzer and produces a **parse tree** based on a **Context-Free Grammar (CFG)**.

Role of Parser

- Checks **syntactic correctness** of the program
- Constructs **Parse Tree / Syntax Tree**
- Detects syntax errors and reports them
- Works based on grammar rules (CFG)

Types of Parsers

1. **Top-Down Parser**
    - Builds tree from root → leaves
    - Example: Recursive Descent Parser
2. **Bottom-Up Parser**
    - Builds tree from leaves → root
    - Example: Shift-Reduce Parser

Context-Free Grammar (CFG)

A CFG is defined as:

G = (V, T, P, S)

Where:

- **V** → Non-terminals
- **T** → Terminals
- **P** → Production rules
- **S** → Start symbol

Example CFG

E → E + T | T
T → T * F | F

F → (E) | id

Interaction Between Parser and CFG

- CFG defines the structure of valid programs
- Parser uses CFG to:
  - Validate syntax
  - Generate parse tree

Neat Diagram (Syntax Analysis Flow)

```
Tokens → Parser → Parse Tree → Semantic Analysis
        ↑
       CFG
```

Key Points for Exam

- Parser ensures grammatical correctness
- CFG defines language syntax
- Parse tree represents structure
- Important for further phases (semantic analysis)

✅ 2. Recursive Descent Parsing (with Example)

Definition / Theory

Recursive Descent Parsing is a **top-down parsing technique** where each non-terminal is implemented as a recursive function.

- Uses **backtracking (if needed)**
- Works well with **LL(1) grammars**

Given Grammar (LL(1))

E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → id | (E)

Input String

id + id * id

Recursive Functions

E() → T(); E'();
E'() → match('+') T() E'() | ε
T() → F(); T'();
T'() → match('*') F() T'() | ε
F() → match(id) | match('(') E() match(')')

Sequence of Function Calls

```
E()
└── T()
    └── F() → match(id)
    └── T'() → ε
└── E'()
    └── match(+)
    └── T()
        └── F() → match(id)
        └── T'()
            └── match(*)
            └── F() → match(id)
            └── T'() → ε
    └── E'() → ε
```

Leftmost Derivation

E
⇒ T E'

⇒ F T' E'
⇒ id T' E'
⇒ id E'
⇒ id + T E'
⇒ id + F T' E'
⇒ id + id T' E'
⇒ id + id * F T' E'
⇒ id + id * id T' E'
⇒ id + id * id

Parsing Concept

- Start from **start symbol (E)**
- Expand using productions
- Match input tokens step-by-step
- If all input is consumed → **Accepted**

Advantages

- Simple to implement
- Easy to debug
- Matches grammar structure

Limitations

- Cannot handle left-recursive grammars
- May require backtracking

Key Points for Exam

- Recursive descent = one function per non-terminal
- Works for LL(1) grammars
- Uses top-down approach
- Shows clear derivation and function calls

1. LL(1) Parsing Table Construction & Parsing Process

Definition / Theory

An **LL(1) parser** is a top-down parser that scans input from **Left to right**, produces a **Leftmost derivation**, and uses **1 lookahead symbol**.

It uses a **predictive parsing table** built using **FIRST and FOLLOW sets**.

Given Grammar

E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → id | (E)

FIRST & FOLLOW (Summary)

- FIRST(E) = { id, (}
- FIRST(E') = { +, ε }
- FIRST(T) = { id, (}
- FIRST(T') = { *, ε }
- FIRST(F) = { id, (}
- FOLLOW(E) = {), $ }
- FOLLOW(E') = {), $ }
- FOLLOW(T) = { +,), $ }
- FOLLOW(T') = { +,), $ }
- FOLLOW(F) = { *, +,), $ }

LL(1) Parsing Table

Non-Terminal	id	+	*	(	)	$
E	T E'			T E'		

E'		+ T E'		ε	ε
T	F T'		F T'		
T'		ε	* F T'	ε	ε
F	id		(E)		

Input String

id + id * id $

Parsing Steps

Stack	Input	Action
E $	id + id * id $	E → T E'
T E' $	id + id * id $	T → F T'
F T' E' $	id + id * id $	F → id
id T' E' $	id + id * id $	match id
T' E' $	+ id * id $	T' → ε
E' $	+ id * id $	E' → + T E'
+ T E' $	+ id * id $	match +
T E' $	id * id $	T → F T'
F T' E' $	id * id $	F → id
id T' E' $	id * id $	match id
T' E' $	* id $	T' → * F T'
* F T' E' $	* id $	match *
F T' E' $	id $	F → id
id T' E' $	id $	match id
T' E' $	$	T' → ε

E' $	$	E' → ε
$	$	Accept

Key Points for Exam

- LL(1) uses **table-driven parsing**
- No backtracking required
- Grammar must be **non-left-recursive & left-factored**
- Successful parsing ends with **stack = $ and input = $**

✅ 2. Shift-Reduce Parsing & Conflict Analysis

Definition / Theory

Shift-Reduce Parsing is a bottom-up parsing technique that constructs the parse tree by **shifting input symbols onto a stack and reducing them using grammar rules**.

Basic Operations

1. **Shift**
   - Push input symbol onto stack
2. **Reduce**
   - Replace RHS of production with LHS
3. **Accept**
   - Parsing successful
4. **Error**
   - Invalid string

Example Grammar

E → E + E | E * E | id

Input String

id + id * id

Parsing Steps

Stack	Input	Action
$	id + id * id $	Shift id
$ id	+ id * id $	Reduce E → id
$ E	+ id * id $	Shift +
$ E +	id * id $	Shift id
$ E + id	* id $	Reduce E → id
$ E + E	* id $	Conflict
$ E + E	* id $	Shift *
$ E + E *	id $	Shift id
$ E + E * id	$	Reduce E → id
$ E + E * E	$	Reduce E → E * E
$ E + E	$	Reduce E → E + E
$ E	$	Accept

Conflicts in Shift-Reduce Parsing

1. Shift-Reduce Conflict

- Parser cannot decide:
  - Shift next symbol OR
  - Reduce current symbols

Example:

E → E + E | E * E

At `E + E *`, ambiguity arises.

---

2. Reduce-Reduce Conflict

- Two possible reductions available

Resolving Conflicts

1. Operator Precedence

- `*` has higher precedence than `+`

2. Associativity

- Left associative: `E → E + E`

3. Grammar Transformation

Unambiguous grammar:

E → E + T | T
T → T * F | F
F → id

Advantages

- Powerful (handles more grammars than LL(1))
- Used in LR parsers

Key Points for Exam

- Bottom-up approach
- Uses stack and input buffer
- Conflicts arise due to ambiguity

- Resolved using precedence & grammar rewriting

1. Construction of SLR Parsing Table (LR(0) Items, Closure & GOTO)

Definition / Theory

An **SLR (Simple LR) parser** is a bottom-up parser that uses **LR(0) items** along with **FOLLOW sets** to construct parsing tables.

- **LR(0) Item**: A production with a dot (•) indicating parsing position
- Example: `E → E • + T`

Given Grammar (Augmented)

E' → E
E → E + T | T
T → T * F | F
F → id

🔹 Step 1: Construct LR(0) Items (Closure & GOTO)

Initial Item (I_0)

I_0:
E' → • E
E → • E + T
E → • T
T → • T * F
T → • F
F → • id

👉 This is obtained using **closure()**

Closure Function

- If `A → α • B β` exists
- Add all productions of B with dot at beginning

GOTO Function

- `goto(I, X)` = set of items after moving dot over symbol X

Example Transitions

GOTO(I_0, id)
I_5:
F → id •

GOTO(I_0, E)
I_1:
E' → E •
E → E • + T

GOTO(I_0, T)
I_2:
E → T •
T → T • * F

GOTO(I_0, F)
I_3:
T → F •

🔹 Step 2: Construct SLR Parsing Table

Rules

- If `A → α • a β` → **Shift**

- If $A \rightarrow \alpha \cdot$ → **Reduce using FOLLOW(A)**
- If $S' \rightarrow S \cdot$ → **Accept**

Sample Table Entries

State	id	+	*	$	E	T	F
I_0	S5				1	2	3
I_1		S6		Acc			
I_2		R2	S7	R2			
I_5		R4	R4	R4			

Key Points for Exam

- Closure → expands non-terminals
- GOTO → transitions between states
- FOLLOW sets used for reductions
- SLR table has **Shift, Reduce, Accept actions**

✅ 2. Difference Between SLR, CLR, and LALR Parsers

Definition / Theory

These are **LR parsers** used in syntax analysis, differing in **power and table size**.

---

1. SLR (Simple LR)

- Uses **LR(0) items + FOLLOW sets**
- Simple but less powerful

- May produce conflicts

---

2. CLR (Canonical LR / LR(1))

- Uses **LR(1) items (with lookahead)**
- Most powerful
- Large parsing tables

---

3. LALR (Look-Ahead LR)

- Combines states of CLR with same core
- Same power as CLR (almost)
- Smaller tables

Comparison Table

Feature	SLR	CLR (LR(1))	LALR
Items Used	LR(0)	LR(1)	LR(1) merged
Lookahead	FOLLOW sets	Explicit	Partial
Parsing Power	Low	High	Medium-High
Table Size	Small	Very Large	Moderate
Conflicts	More	Very Few	Few
Implementation	Easy	Complex	Moderate

Example Insight

Grammar:

S → L = R | R
L → * R | id
R → L

- **SLR** → conflict occurs
- **CLR** → no conflict (uses lookahead)
- **LALR** → resolves with merged states

Parsing Power Analysis

- **SLR**
  - Cannot handle complex grammars
- **CLR**
  - Handles almost all deterministic CFGs
- **LALR**
  - Balance between power and efficiency
  - Used in tools like **YACC**

Key Points for Exam

- SLR → simple but limited
- CLR → powerful but large tables
- LALR → practical and widely used
- Trade-off between **power vs memory**

1. Handling Ambiguity (Dangling-Else Problem) in LR Parsing

Definition / Theory

An **ambiguous grammar** is one that produces more than one parse tree for the same input string.
The **dangling-else problem** is a classic ambiguity in conditional statements.

Ambiguous Grammar

S → if E then S | if E then S else S | other

Problem Explanation

Input:

if E1 then if E2 then S1 else S2

Two interpretations:

1. `else` matches **inner if**
2. `else` matches **outer if**

👉 This creates ambiguity.

How LR Parsing Handles It

In **shift-reduce parsing**, a conflict occurs:

- **Shift** → associate `else` with nearest `if`
- **Reduce** → associate with earlier `if`

Resolution Strategy

✅ Rule:

"Else is matched with the nearest unmatched if"

👉 This is implemented by **preferring SHIFT over REDUCE**

Unambiguous Grammar (Solution)

S → matched | unmatched

matched → if E then matched else matched | other
unmatched → if E then S | if E then matched else unmatched

Parsing Steps (Simplified)

Input:

if E1 then if E2 then S1 else S2

Stack Behavior:

Stack	Input	Action
if E then	if E then S1 else S2	Shift
if E then if E then	S1 else S2	Shift
if E then if E then S1	else S2	Reduce S1
...	else S2	**Shift else**
...	S2	Attach else to inner if

Key Idea

- LR parser **chooses SHIFT**
- Ensures `else` binds to **nearest if**

✅ 2. Syntax-Directed Definitions (SDD) & Dependency Graph

Definition / Theory

A **Syntax-Directed Definition (SDD)** is a CFG with **attributes** and **semantic rules** used to compute values (like types, expressions, etc.).

Types of Attributes

1. **Synthesized Attributes**
    - Computed from children nodes
    - Used in bottom-up parsing
2. **Inherited Attributes**
    - Passed from parent/siblings

Dependency Graph

- A directed graph showing **dependencies between attributes**
- Node → attribute
- Edge → dependency

Example SDD

E → E1 + T { E.val = E1.val + T.val }
E → T { E.val = T.val }
T → num { T.val = num.val }

Dependency Graph Construction

For:

E → E1 + T

Dependencies:

- E1.val → E.val
- T.val → E.val

Graph Representation

```
E1.val ----\
           → E.val
T.val  -----/
```

Evaluation Order

- Must follow **topological order**
- No cycles allowed

Steps to Evaluate SDD

1. Construct parse tree
2. Build dependency graph
3. Perform **topological sorting**
4. Evaluate attributes in order

Valid Evaluation Order (Example)

num.val → T.val → E1.val → E.val

Types of SDD Based on Evaluation

1. **S-Attributed**
    - Only synthesized attributes
    - Easy (bottom-up)
2. **L-Attributed**
    - Allows inherited attributes
    - Evaluated left-to-right

1. Intermediate Code Representations (Syntax Tree, DAG, TAC)

Definition / Theory

Intermediate Code (IC) is a machine-independent representation of source code generated by the compiler after semantic analysis.
It helps in **optimization and code generation**.

Types of Intermediate Representations

🔹 1. Syntax Tree (Abstract Syntax Tree - AST)

Description

- Tree representation of program structure
- Internal nodes → operators
- Leaf nodes → operands

Example

Expression:

a + b * c

Tree:

```
   (+)
  /   \
 a     (*)
      /   \
     b     c
```

Advantages

- Represents hierarchical structure
- Easy for semantic analysis

Disadvantages

- Does not eliminate redundant computations
- Less efficient for optimization

🔹 2. Directed Acyclic Graph (DAG)**

Description

- Similar to syntax tree but **eliminates common subexpressions**

- Nodes can have multiple parents

Example

a + b + a + b

DAG shares `(a + b)` instead of recomputing.

Advantages

- Detects common subexpressions
- Reduces redundant computations
- Improves optimization

Disadvantages

- More complex than syntax trees

🔹 3. Three-Address Code (TAC)**

Description

- Linear representation
- Each instruction has at most 3 operands

Example

t1 = b * c
t2 = a + t1

Forms of TAC

- Quadruples
- Triples
- Indirect triples

Advantages

- Easy for code generation
- Suitable for optimization
- Machine-independent

Disadvantages

- Requires more temporary variables

🔹 Comparison Table

Feature	Syntax Tree	DAG	TAC
Structure	Tree	Graph	Linear
Optimization	Low	High	Medium–High
Redundancy Removal	No	Yes	Partial
Complexity	Simple	Complex	Moderate
Suitability	Analysis	Optimization	Code Generation

Conclusion

- **Syntax Tree** → best for structure representation
- **DAG** → best for optimization
- **TAC** → best for code generation

2. Syntax-Directed Translation (SDT) for L-Attributed Definitions

Definition / Theory

A **Syntax-Directed Translation (SDT)** is a CFG with **semantic actions embedded within productions** to perform translation during parsing.

An **L-attributed definition** allows attributes to be evaluated **left-to-right**, making it suitable for top-down parsing.

L-Attributed Definition Rules

- Inherited attributes depend only on:
  - Parent node
  - Left siblings

Example Grammar with SDT

E → T E' { E.val = T.val + E'.val }
E' → + T E'1 { E'.val = T.val + E'1.val }
E' → ε { E'.val = 0 }
T → num { T.val = num.val }

Translation Scheme (with Actions)

E → T { E'.inh = T.val } E'
E' → + T { E'1.inh = E'.inh + T.val } E'1
E' → ε { E'.val = E'.inh }

Why SDT is Effective

✅ 1. Direct Translation

- Actions are executed during parsing
- No need for separate tree traversal

✅ 2. Efficient Evaluation

- Attributes computed in **single pass**

✅ 3. Suitable for Top-Down Parsing

- Works naturally with **recursive descent parser**

Example Evaluation

Input:

3 + 4 + 5

Evaluation:

E.val = 3 + 4 + 5 = 12

Advantages

- Simple and efficient
- Supports real-time translation
- No need for dependency graph explicitly

Limitations

- Restricted to L-attributed definitions
- Complex dependencies are hard to handle

1. Type Checking & Backpatching in Intermediate Code Generation

Definition / Theory

- **Type Checking**: Process of verifying that operands and operations in a program are **type compatible**.
- **Backpatching**: Technique used to handle **forward jumps** in intermediate code by filling missing addresses later.

Both are essential in **intermediate code generation** to ensure correctness and proper control flow.

🔹 Type Checking

Purpose

- Detect **type errors** during compilation
- Ensure operations are semantically valid

Types of Type Checking

1. **Static Type Checking**
   - Done at compile time
   - Example: `int + float` → type conversion required
2. **Dynamic Type Checking**
   - Done at runtime

Example

int a;
float b;
a = b + 2;

👉 Compiler checks:

- `b + 2` → float
- Assigning float to int → requires conversion or error

Role in Intermediate Code

- Inserts **type conversions**
- Prevents invalid operations
- Ensures correctness before code generation

🔹 Backpatching

Definition

Backpatching is the process of **filling incomplete jump addresses** in intermediate code after the target labels are known.

Need

- Used in:
  - `if`, `if-else`
  - `while`, `for`
- Target address not known at the time of generation

Example

if (a < b) goto L1
goto L2
L1: x = 1
L2: x = 2

Initially:

if (a < b) goto _
goto _

Later:

- Fill addresses using **backpatching**

Backpatching Functions

- **makelist(i)** → create new list
- **merge(p1, p2)** → combine lists
- **backpatch(p, target)** → fill addresses

Role in Ensuring Correctness

Technique	Role
Type Checking	Prevents semantic errors
Backpatching	Ensures correct control flow
Combined Effect	Produces valid intermediate code

---

2. Top-Down vs Bottom-Up Parsing & Preprocessing Steps

Definition / Theory

- **Top-Down Parsing**: Starts from root (start symbol) and builds parse tree downward
- **Bottom-Up Parsing**: Starts from input and builds parse tree upward

---

🔹 Top-Down Parsing

Examples

- Recursive Descent
- LL(1) Parser

Working

- Expands non-terminals
- Uses **leftmost derivation**

Preprocessing Steps

- Eliminate **left recursion**
- Perform **left factoring**

- Compute **FIRST & FOLLOW sets**

🔹 Bottom-Up Parsing

Examples

- Shift-Reduce
- SLR, LR, LALR

Working

- Reduces input string to start symbol
- Uses **rightmost derivation (in reverse)**

Preprocessing Steps

- Augment grammar
- Construct **LR(0)/LR(1) items**
- Build **parsing tables (ACTION & GOTO)**

🔹 Comparison Table

Feature	Top-Down Parsing	Bottom-Up Parsing
Approach	Root → Leaves	Leaves → Root
Derivation	Leftmost	Rightmost (reverse)
Backtracking	May be required	Not required
Grammar Restrictions	Cannot handle left recursion	Handles wider grammars
Complexity	Simple	Complex
Power	Less	More

🔹 Example Insight

Grammar:

E → E + T | T

- Top-down ❌ fails (left recursion)
- Bottom-up ✅ works

Conclusion

- Top-down → simple, but limited
- Bottom-up → powerful, widely used in compilers

---

1. Runtime Environment for Procedure Calls & Activation Records

Definition / Theory

A **runtime environment** is the system that manages **memory, procedure calls, and execution flow** during program execution.

An **Activation Record (AR)** (or stack frame) is a data structure used to store information about a function call.

Structure of Activation Record

Typical components:

```
+----------------------+
| Return Value         |
| Parameters           |
| Return Address       |
```

```
| Control Link        |
| Access Link         |
| Local Variables     |
| Temporary Variables |
+---------------------+
```

Explanation of Fields

- **Return Value** → Stores function result
- **Parameters** → Input arguments
- **Return Address** → Where to return after execution
- **Control Link** → Points to caller's AR
- **Access Link** → Used for nested scopes
- **Local Variables** → Function variables
- **Temporaries** → Intermediate computations

Procedure Call Mechanism

1. Caller pushes arguments
2. New activation record is created
3. Control transferred to called function
4. Function executes
5. Result returned
6. Activation record removed

Parameter Passing Mechanisms

🔹 1. Call by Value

- Copy of value passed
- Changes do not affect original

🔹 2. Call by Reference

- Address passed
- Changes affect original variable

🔹 **3. Call by Name (Conceptual)**

- Expression evaluated when used

Example

void fun(int x) { x = x + 10; }

- Call by value → original unchanged
- Call by reference → original updated

Role in Runtime Environment

- Manages function execution
- Supports recursion
- Maintains scope and variable access

2. Code Optimization using Data-Flow Analysis

Definition / Theory

Data-flow analysis is a technique used by compilers to gather information about the **flow of data across program statements** to optimize code.

Goal of Optimization

- Reduce execution time
- Minimize memory usage
- Remove redundant computations

Basic Idea

- Analyze how variables are defined and used
- Apply transformations to improve efficiency

Example Code (Before Optimization)

```
a = b + c
d = b + c
e = a + d
```

Optimization Technique: Common Subexpression Elimination

After Optimization

```
t1 = b + c
a = t1
d = t1
e = a + d
```

👉 Eliminates repeated computation

Data-Flow Analysis Concepts

🔹 **1. Reaching Definitions**

- Determines which definitions reach a point

🔹 **2. Live Variable Analysis**

- Determines variables needed in future

🔹 **3. Available Expressions**

- Identifies reusable expressions

Algorithm (Simplified Steps)

1. Divide program into **basic blocks**
2. Construct **Control Flow Graph (CFG)**
3. Initialize IN and OUT sets
4. Apply iterative equations:

   $IN[n] = \cup OUT[p]$ (for all predecessors p)
   $OUT[n] = GEN[n] \cup (IN[n] - KILL[n])$

5. Repeat until no change (fixed point)

Illustration

Before:

- Repeated computation
- More instructions

After:

- Fewer operations
- Faster execution

Other Optimizations using Data-Flow

- Dead Code Elimination
- Constant Propagation
- Copy Propagation

Advantages

- Improves performance
- Reduces redundancy
- Enhances code quality

1. Code Generation for Arithmetic Expressions (Register Allocation & Instruction Selection)

Definition / Theory

Code Generation is the final phase of a compiler that converts intermediate code (like TAC) into **target machine code**.

It involves:

- **Instruction Selection** → choosing machine instructions
- **Register Allocation** → assigning variables to CPU registers

---

Steps in Code Generation

1. Convert expression into **Intermediate Code (TAC)**
2. Select appropriate **machine instructions**
3. Allocate **registers efficiently**
4. Generate final target code

Example Expression

a = b + c * d

Step 1: Three-Address Code (TAC)

```
t1 = c * d
t2 = b + t1
a = t2
```

Step 2: Instruction Selection

Target machine (example):

```
LOAD R1, c
MUL  R1, d
LOAD R2, b
ADD  R2, R1
STORE a, R2
```

Register Allocation

Goal:

- Minimize number of registers
- Reduce memory access

Strategy:

- Use registers for frequently used variables
- Reuse registers when values are no longer needed

Example Allocation:

- R1 → stores `c*d`
- R2 → stores `b + t1`

Register Allocation Techniques

1. **Simple Allocation**
    - Assign registers sequentially
2. **Graph Coloring (Advanced)**
    - Interference graph used
    - Efficient allocation

Instruction Selection Techniques

- Use efficient machine instructions

- Prefer:
    - `MUL` instead of repeated addition
    - Combined instructions if available

Optimized Code

```
LOAD R1, c
MUL  R1, d
ADD  R1, b
STORE a, R1
```

👉 Reduced one register → more efficient

2. Peephole Optimization Module

Definition / Theory

Peephole Optimization is a local optimization technique that examines a small sequence of instructions (a “peephole”) and replaces inefficient patterns with better ones.

Goal

- Remove redundant instructions
- Improve execution speed
- Reduce code size

Working of Peephole Optimizer

1. Scan small instruction window
2. Identify inefficient patterns
3. Replace with optimized instructions

Common Techniques

🔹 1. Redundant Instruction Elimination

Before

```
LOAD R1, a
LOAD R1, a
```

After

```
LOAD R1, a
```

🔹 2. Constant Folding

Before

```
ADD R1, 2, 3
```

After

```
LOAD R1, 5
```

🔹 3. Strength Reduction

Before

```
MUL R1, x, 2
```

After

```
ADD R1, x, x
```

🔹 4. Dead Code Elimination

Before

```
x = 10
x = 20
```

After

```
x = 20
```

🔹 5. Jump Optimization

Before

GOTO L1
L1: GOTO L2

After

GOTO L2

Algorithm (Simplified)

1. Take window of k instructions
2. Match patterns
3. Replace with optimized sequence
4. Slide window and repeat

Advantages

- Simple and effective
- Works after code generation
- Improves performance locally

Limitations

- Only local optimization
- Cannot handle global improvements

1. Role of Parser and Context-Free Grammars (CFG) in Syntax Analysis

Definition / Theory

Syntax Analysis (Parsing) is the compiler phase that checks whether a sequence of tokens follows the **grammatical rules** of a programming language.

A **Parser** is a program that takes tokens from the lexical analyzer and builds a **parse tree** using a **Context-Free Grammar (CFG)**.

Role of Parser

- Verifies **syntactic correctness** of input
- Constructs **Parse Tree / Abstract Syntax Tree (AST)**
- Detects and reports **syntax errors**
- Provides structure for **semantic analysis**

Context-Free Grammar (CFG)

A CFG is defined as:

G = (V, T, P, S)

Where:

- **V** → Non-terminals
- **T** → Terminals
- **P** → Production rules
- **S** → Start symbol

Example CFG

E → E + T | T
T → T * F | F
F → (E) | id

How Parser Uses CFG

- CFG defines valid sentence structure
- Parser applies productions to:
    - Validate input
    - Generate parse tree

Types of Parsers

1. **Top-Down Parser**
    - Builds tree from root → leaves
    - Example: Recursive Descent, LL(1)
2. **Bottom-Up Parser**
    - Builds tree from leaves → root
    - Example: Shift-Reduce, LR

2. LL(1) vs LR Parsers & Recursive Descent Parsing

Definition / Theory

- **LL(1) Parser** → Top-down parser using **1 lookahead symbol**
- **LR Parser** → Bottom-up parser using **shift-reduce technique**

🔹 Differences Between LL(1) and LR Parsers

Feature	LL(1) Parser	LR Parser
Parsing Type	Top-Down	Bottom-Up
Derivation	Leftmost	Rightmost (reverse)
Grammar Restrictions	No left recursion	Handles left recursion
Backtracking	Not required	Not required
Complexity	Simple	Complex

Parsing Power	Less	More
Table Size	Small	Large

🔹 Recursive Descent Parsing

Definition

A **Recursive Descent Parser** is a top-down parser where each non-terminal is implemented as a **recursive function**.

Working

- Start from start symbol
- Apply productions using functions
- Match input tokens step-by-step

Example Grammar (LL(1))

E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → id | (E)

Recursive Functions

E() → T(); E'();
E'() → match('+') T() E'() | ε
T() → F(); T'();
T'() → match('*') F() T'() | ε
F() → match(id) | match('(') E() match(')')

Example Input

id + id

Function Call Flow

```
E()
└── T()
    └── F() → match(id)
└── E'()
    └── match(+)
    └── T()
        └── F() → match(id)
    └── E'() → ε
```

Advantages

- Easy to implement
- Matches grammar structure
- Good for small languages

Limitations

- Cannot handle left-recursive grammars
- May need grammar transformation